

PostgreSQL Compliance Considerations (GDPR, SOC2, HIPAA)

Table of Contents

- 1. [Learning Objectives](#)
- 2. [Compliance Framework Overview](#)
- 3. [GDPR Compliance](#)
- 4. [SOC 2 Compliance](#)
- 5. [HIPAA Compliance](#)
- 6. [PCI-DSS Compliance](#)
- 7. [PostgreSQL Compliance Checklist](#)
- 8. [Data Classification and Labeling](#)
- 9. [Data Retention and Deletion](#)
- 10. [Compliance Architecture Patterns](#)
- 11. [Common Mistakes](#)
- 12. [Best Practices](#)
- 13. [Interview Questions](#)
- 14. [Exercises and Solutions](#)

Learning Objectives

By the end of this module, you will be able to:

- Map compliance requirements to specific PostgreSQL configurations
- Implement GDPR right-to-erasure in PostgreSQL schemas
- Configure PostgreSQL for SOC2 audit evidence
- Apply HIPAA minimum necessary standards to PostgreSQL access control
- Design a data classification scheme within PostgreSQL
- Build a compliance checklist for PostgreSQL deployments

Compliance Framework Overview

COMPLIANCE FRAMEWORK COMPARISON:

Framework	Scope	Key Concern	Penalty
GDPR	EU personal data	Privacy rights Data subject	€20M or 4% revenue
SOC 2	Service orgs handling customer data	Security, availability, confidentiality	Loss of contracts, business trust
HIPAA	US healthcare data (PHI)	PHI protection Minimum access	\$100-\$50,000/ violation; criminal
PCI-DSS	Card payment	CHD protection	Fines, loss of

	data (CHD)	Encryption	card processing
ISO 27001	Information	ISMS framework	Certification loss
	security mgmt		

GDPR Compliance

Key GDPR Articles Relevant to PostgreSQL

Article	Requirement	PostgreSQL Implementation
Art. 5	Data minimization	Column-level encryption, separate PII tables
Art. 17	Right to erasure	DELETE + cascade + anonymization
Art. 20	Data portability	Structured export (JSON/CSV)
Art. 25	Privacy by design	RLS, encryption by default
Art. 30	Records of processing	Audit logging
Art. 32	Technical measures	Encryption, access control, backups
Art. 33/34	Breach notification	Audit log + alerting

Right to Erasure (Art. 17) Implementation

```
-- Complete user deletion preserving referential integrity
-- Strategy: Anonymize rather than hard-delete (preserve audit history)

-- Option 1: Hard Delete (simple but breaks foreign keys and audit)
DELETE FROM users WHERE id = :user_id;
-- Problem: Cascades to orders, audit_log, etc.

-- Option 2: Pseudonymization (GDPR-preferred approach)
-- Replace PII with pseudonymous values, preserve transactional data

CREATE OR REPLACE FUNCTION gdpr_erase_user(p_user_id INT)
RETURNS void AS $$
DECLARE
    anon_suffix TEXT := 'ERASED_' || p_user_id::text || '_' || to_char(now(),
'YYYYMMDD');
BEGIN
    -- Anonymize the user record (preserve for audit, remove PII)
    UPDATE users SET
        email = anon_suffix || '@erased.invalid',
        name = 'ERASED USER ' || p_user_id,
        phone = NULL,
        address = NULL,
        date_of_birth = NULL,
        ssn_encrypted = NULL,
        -- Keep: id, account_type, created_at (for audit), status
```

```

        status = 'ERASED',
        erased_at = now(),
        erased_by = current_user
WHERE id = p_user_id;

-- Delete PII from related tables
DELETE FROM user_pii_detail WHERE user_id = p_user_id;
DELETE FROM user_sessions WHERE user_id = p_user_id;
DELETE FROM email_preferences WHERE user_id = p_user_id;

-- Anonymize orders (keep for financial records, strip PII)
UPDATE orders SET
    shipping_address = 'ERASED',
    billing_address = 'ERASED'
WHERE user_id = p_user_id;

-- Log the erasure (required by GDPR for accountability)
INSERT INTO gdpr_erasure_log (user_id, erased_at, erased_by, tables_affected)
VALUES (p_user_id, now(), current_user, ARRAY['users', 'user_pii_detail',
'user_sessions', 'orders']);

RAISE NOTICE 'GDPR erasure completed for user %', p_user_id;
END;
$$ LANGUAGE plpgsql;

-- Usage
SELECT gdpr_erase_user(42);

```

Data Subject Access Request (DSAR)

```

-- Export all personal data for a user (Art. 15/20 – Right to access/portability)
CREATE OR REPLACE FUNCTION gdpr_export_user_data(p_user_id INT)
RETURNS JSONB AS $$
DECLARE
    user_data JSONB;
BEGIN
    SELECT jsonb_build_object(
        'user_info', row_to_json(u),
        'orders', (
            SELECT jsonb_agg(row_to_json(o))
            FROM orders o WHERE o.user_id = p_user_id
        ),
        'addresses', (
            SELECT jsonb_agg(row_to_json(a))
            FROM addresses a WHERE a.user_id = p_user_id
        ),
        'audit_history', (
            SELECT jsonb_agg(jsonb_build_object(
                'action', action,
                'timestamp', created_at,
                'ip_address', ip_address
            ))

```

```

        ))
        FROM user_activity_log WHERE user_id = p_user_id
        ORDER BY created_at DESC
    ),
    'exported_at', now(),
    'export_version', '1.0'
)
INTO user_data
FROM users u
WHERE u.id = p_user_id;

-- Log the data export
INSERT INTO dsar_log (user_id, exported_at, tables_included)
VALUES (p_user_id, now(), ARRAY['users', 'orders', 'addresses',
'user_activity_log']);

RETURN user_data;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

```

Privacy by Design (Art. 25) Schema Pattern

```

-- Separate PII from transactional data
-- Transactional data: long retention (7 years for financial)
-- PII: delete when user requests or retention period ends

-- Core table: no PII
CREATE TABLE orders (
    id BIGSERIAL PRIMARY KEY,
    user_id INT NOT NULL,          -- Reference, not PII itself
    product_id INT NOT NULL,
    quantity INT,
    amount NUMERIC(10,2),
    status TEXT,
    created_at TIMESTAMPTZ DEFAULT now()
);

-- PII satellite table: separate retention and deletion
CREATE TABLE order_pii (
    order_id BIGINT PRIMARY KEY REFERENCES orders(id) ON DELETE CASCADE,
    shipping_name TEXT,
    shipping_address TEXT,
    billing_address TEXT,
    pii_deleted_at TIMESTAMPTZ    -- NULL = active, set when user erased
);

-- User PII table: purge on erasure request
CREATE TABLE user_pii (
    user_id INT PRIMARY KEY REFERENCES users(id) ON DELETE CASCADE,
    full_name TEXT,
    email TEXT UNIQUE,

```

```
phone TEXT,  
date_of_birth DATE,  
national_id_encrypted BYTEA  
);
```

Data Retention Policy

```
-- Implement automatic data retention  
CREATE OR REPLACE FUNCTION enforce_data_retention()  
RETURNS void AS $$  
BEGIN  
    -- Delete user sessions older than 30 days  
    DELETE FROM user_sessions  
    WHERE created_at < now() - INTERVAL '30 days';  
  
    -- Anonymize audit logs older than 2 years  
    UPDATE audit_log SET  
        ip_address = NULL,  
        user_agent = NULL  
    WHERE created_at < now() - INTERVAL '2 years'  
        AND ip_address IS NOT NULL;  
  
    -- Mark users for erasure whose accounts are expired > 1 year  
    UPDATE users SET status = 'PENDING_ERASURE'  
    WHERE account_deleted_at < now() - INTERVAL '1 year'  
        AND status = 'DELETED';  
  
    RAISE NOTICE 'Data retention enforcement completed at %', now();  
END;  
$$ LANGUAGE plpgsql;  
  
-- Schedule via pg_cron  
SELECT cron.schedule('data-retention', '0 2 * * 0', 'SELECT  
enforce_data_retention()');
```

SOC 2 Compliance

SOC 2 is organized around Trust Service Criteria (TSC):

Security (CC6, CC7)

```
-- CC6.1: Logical access controls  
-- Implementation: Role-based access control  
SELECT rolname, rolsuper, rolcanlogin, rolconlimit  
FROM pg_roles  
WHERE rolname NOT LIKE 'pg_%'  
ORDER BY rolname;  
  
-- CC6.2: Authentication controls
```

```

-- Verify SCRAM-SHA-256 for all users
SELECT rolname FROM pg_authid
WHERE rolcanlogin = true
    AND (passwd NOT LIKE 'SCRAM-SHA-256%' OR passwd IS NULL);
-- Zero rows = compliant

-- CC6.6: Security incidents
-- Audit log for access monitoring (see 06_auditing.md)

-- CC6.7: Data transmission protection
-- Verify SSL requirement in pg_hba.conf
SELECT type, database, user_name, auth_method
FROM pg_hba_file_rules
WHERE type = 'host' AND auth_method != 'reject';
-- All 'host' entries should be 'hostssl' in production

```

Availability (A1)

```

-- A1.1: System availability monitoring
-- Check uptime
SELECT pg_postmaster_start_time(),
       now() - pg_postmaster_start_time() AS uptime;

-- A1.2: Backup and recovery
-- Verify last backup completed (from pgBackRest)
-- See 15_Backup_Recovery/06_pgbackrest.md

-- A1.3: Capacity planning
SELECT pg_size_pretty(pg_database_size('mydb')) AS db_size;
SELECT pg_size_pretty(SUM(pg_total_relation_size(schemaname||'.'||tablename))) AS
total_tables_size
FROM pg_tables WHERE schemaname = 'public';

```

Confidentiality (C1)

```

-- C1.1: Classification and labeling
-- Implement via column comments or a metadata table
COMMENT ON COLUMN users.email IS 'GDPR:PII CLASSIFICATION:CONFIDENTIAL';
COMMENT ON COLUMN users.password_hash IS 'CLASSIFICATION:RESTRICTED';
COMMENT ON COLUMN orders.amount IS 'CLASSIFICATION:INTERNAL';

-- Query classified columns
SELECT col.table_name, col.column_name, pgd.description AS classification
FROM information_schema.columns col
JOIN pg_catalog.pg_statio_all_tables st ON st.relname = col.table_name
JOIN pg_catalog.pg_description pgd ON pgd.objoid = st.relid
    AND pgd.objsubid = col.ordinal_position
WHERE col.table_schema = 'public'

```

```
    AND pgd.description LIKE '%CLASSIFICATION%'
ORDER BY col.table_name;
```

SOC 2 Evidence Collection

```
-- Evidence queries for SOC 2 auditor

-- E1: List all users with login privileges
SELECT rolname AS username,
       rolsuper AS is_superuser,
       rolconnlimit AS connection_limit,
       rolvaliduntil AS password_expires
FROM pg_roles
WHERE rolcanlogin = true
ORDER BY rolname;

-- E2: List all role memberships
SELECT r.rolname AS role, m.rolname AS member
FROM pg_auth_members am
JOIN pg_roles r ON r.oid = am.roleid
JOIN pg_roles m ON m.oid = am.member
ORDER BY r.rolname, m.rolname;

-- E3: SSL configuration
SHOW ssl;
SHOW ssl_min_protocol_version;
SHOW ssl_ciphers;

-- E4: Audit logging configuration
SHOW log_connections;
SHOW log_disconnections;
SHOW pgaudit.log;
```

HIPAA Compliance

HIPAA protects Protected Health Information (PHI) in the US healthcare sector.

Administrative Safeguards

```
-- Access control policy: minimum necessary
-- Only grant access to PHI that role requires

-- HIPAA role model
CREATE ROLE phi_read;           -- Read PHI (doctors, authorized staff)
CREATE ROLE phi_write;          -- Write PHI (clinical staff)
CREATE ROLE phi_admin;          -- Manage PHI access (privacy officer)
CREATE ROLE phi_audit;          -- Audit PHI access (compliance)

-- Apply RLS for minimum necessary (each user sees only their patients' records)
```

```

ALTER TABLE patient_records ENABLE ROW LEVEL SECURITY;
ALTER TABLE patient_records FORCE ROW LEVEL SECURITY;

CREATE POLICY provider_sees_own_patients ON patient_records
  FOR ALL TO phi_read
  USING (
    patient_id IN (
      SELECT patient_id FROM provider_patient_relationships
      WHERE provider_id = current_setting('app.provider_id')::INT
      AND active = true
    )
  );

```

Technical Safeguards

```

-- Audit controls: Track access to PHI
-- Configure pgaudit for all PHI tables
ALTER TABLE patient_records SET (pgaudit.log = 'all');
ALTER TABLE patient_demographics SET (pgaudit.log = 'all');
ALTER TABLE medical_history SET (pgaudit.log = 'all');
ALTER TABLE prescriptions SET (pgaudit.log = 'all');

-- Automatic logoff: terminate idle sessions
-- postgresql.conf: idle_in_transaction_session_timeout = 900000 (15 min)

-- Encryption: encrypt PHI at rest
-- Use pgcrypto for sensitive fields
-- Use filesystem encryption (LUKS/EBS) for data-at-rest

-- Transmission security: require SSL
-- All pg_hba.conf entries must be 'hostssl'

```

HIPAA PHI Inventory

```

-- Track which columns contain PHI
CREATE TABLE phi_inventory (
  schema_name TEXT,
  table_name TEXT,
  column_name TEXT,
  phi_category TEXT, -- 'PHI', 'de-identified', 'operational'
  encryption_method TEXT,
  last_reviewed TIMESTAMPTZ,
  reviewed_by TEXT
);

INSERT INTO phi_inventory VALUES
  ('public', 'patients', 'name', 'PHI', 'column_encryption', now(),
  'privacy_officer'),
  ('public', 'patients', 'dob', 'PHI', 'column_encryption', now(),

```



```
'privacy_officer'),
    ('public', 'patients', 'ssn', 'PHI', 'column_encryption', now()),
'privacy_officer'),
    ('public', 'appointments', 'appointment_date', 'PHI', 'none', now()),
'privacy_officer'),
    ('public', 'lab_results', 'result_value', 'PHI', 'column_encryption', now()),
'privacy_officer');
```

PCI-DSS Compliance

PCI-DSS protects cardholder data (CHD).

```
-- PCI-DSS Requirement 3: Protect stored cardholder data
-- Never store full PAN (Primary Account Number) in plaintext

CREATE TABLE payment_tokens (
    id SERIAL PRIMARY KEY,
    user_id INT NOT NULL,
    -- COMPLIANT: Store only token, last 4, and masked PAN
    payment_token TEXT UNIQUE NOT NULL, -- Gateway token, not actual card
    masked_pan TEXT NOT NULL,           -- XXXX-XXXX-XXXX-1234
    last_four CHAR(4) NOT NULL,
    card_type TEXT NOT NULL,
    expiry_month INT,
    expiry_year INT,
    -- NOT stored: Full PAN, CVV/CVC (NEVER store these)
    created_at TIMESTAMPTZ DEFAULT now()
);

-- PCI-DSS Requirement 7: Restrict access to CHD
REVOKE ALL ON payment_tokens FROM PUBLIC;
GRANT SELECT, INSERT ON payment_tokens TO payment_processor_role;
-- No UPDATE of payment token (immutable)
-- No DELETE except by retention process

-- PCI-DSS Requirement 10: Track and monitor all access
ALTER TABLE payment_tokens SET (pgaudit.log = 'all');

-- PCI-DSS Requirement 6: Protect against vulnerabilities
-- Use parameterized queries – enforce at application level
-- PostgreSQL doesn't log parameters in pgaudit by default
-- Enable pgaudit.log_parameter = off for PAN tables (don't log card data)
```

PostgreSQL Compliance Checklist

AUTHENTICATION & ACCESS CONTROL:

- [] All remote connections use SSL (hostssl, not host)
- [] Password authentication uses scram-sha-256 (not md5 or trust)
- [] Default passwords changed from vendor defaults

```
[ ] Superuser access restricted (no shared superuser accounts)
[ ] Principle of least privilege enforced (role-based)
[ ] Service accounts have no unnecessary privileges
[ ] Password expiration configured for human accounts
[ ] Connection limits set per role
```

ENCRYPTION:

```
[ ] SSL/TLS enabled (ssl = on)
[ ] TLS 1.2 minimum (ssl_min_protocol_version = TLSv1.2)
[ ] PII/sensitive columns encrypted with pgcrypto
[ ] Backups encrypted
[ ] Key management process documented
```

AUDITING:

```
[ ] pgaudit installed and configured
[ ] DDL changes audited (pgaudit.log includes 'ddl')
[ ] Privileged user actions audited (audit all DBA activity)
[ ] Sensitive table access logged (object auditing)
[ ] Connection events logged (log_connections = on)
[ ] Authentication failures logged
[ ] Logs retained per compliance requirement (1-7 years)
[ ] Logs shipped to external, tamper-evident storage
```

AVAILABILITY:

```
[ ] High availability configured (Patroni or streaming replication)
[ ] Backup and recovery tested (not just configured)
[ ] RTO/RPO documented and tested
[ ] Disaster recovery plan documented
```

DATA MANAGEMENT:

```
[ ] Data classification scheme implemented
[ ] Data retention policy implemented and automated
[ ] Data deletion/anonymization procedures tested (GDPR erasure)
[ ] PII inventory maintained
[ ] Data export capability for subject access requests
```

VULNERABILITY MANAGEMENT:

```
[ ] PostgreSQL version up-to-date (within 2 minor versions)
[ ] Security patches applied within defined SLA
[ ] pg_hba.conf reviewed regularly (remove stale entries)
[ ] Role/privilege audit conducted quarterly
```

Data Classification and Labeling

```
-- Implement data classification as column comments
-- Comment format: CLASSIFICATION:<level> GDPR:<yes/no> HIPAA:<yes/no>

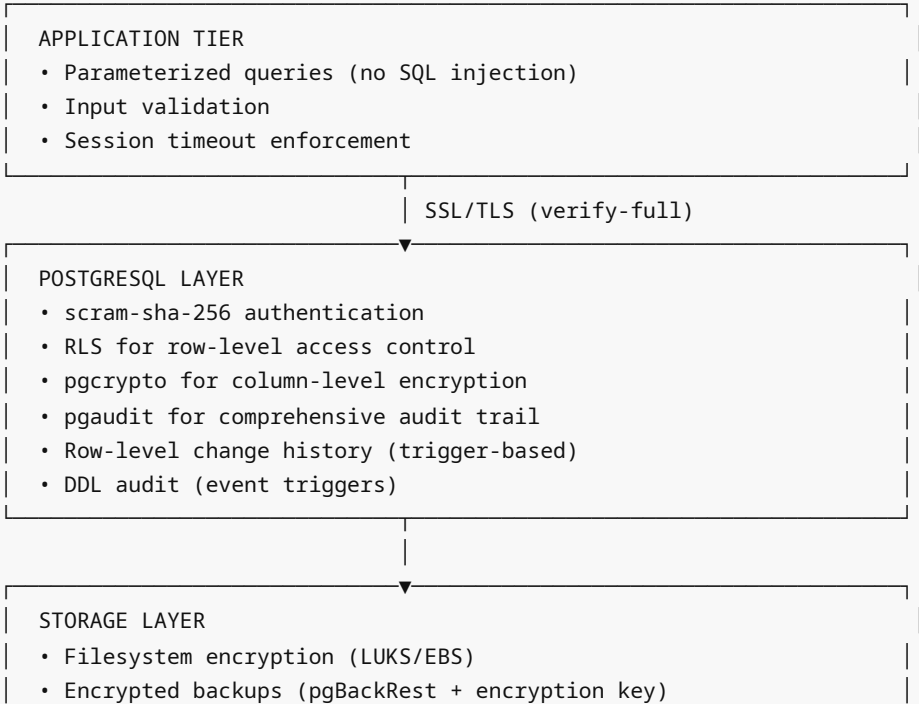
-- Apply classifications
COMMENT ON COLUMN users.id IS 'CLASSIFICATION:INTERNAL';
COMMENT ON COLUMN users.email IS 'CLASSIFICATION:CONFIDENTIAL GDPR:YES';
```

```
COMMENT ON COLUMN users.password_hash IS 'CLASSIFICATION:RESTRICTED';
COMMENT ON COLUMN users.name IS 'CLASSIFICATION:CONFIDENTIAL GDPR:YES';
COMMENT ON COLUMN users.date_of_birth IS 'CLASSIFICATION:CONFIDENTIAL GDPR:YES HIPAA:YES';
COMMENT ON COLUMN users.ssn IS 'CLASSIFICATION:RESTRICTED GDPR:YES HIPAA:YES';
COMMENT ON COLUMN orders.total_amount IS 'CLASSIFICATION:INTERNAL';
COMMENT ON COLUMN patients.diagnosis IS 'CLASSIFICATION:RESTRICTED HIPAA:YES';

-- Query classified columns
SELECT
    c.table_schema,
    c.table_name,
    c.column_name,
    pgd.description AS data_classification
FROM information_schema.columns c
JOIN pg_catalog.pg_statio_all_tables st ON st.relname = c.table_name
    AND st.schemaname = c.table_schema
JOIN pg_catalog.pg_attribute pa ON pa.attrelid = st.relid
    AND pa.attname = c.column_name
LEFT JOIN pg_catalog.pg_description pgd ON pgd.objoid = st.relid
    AND pgd.objsubid = pa.attnum
WHERE c.table_schema = 'public'
    AND pgd.description LIKE '%CLASSIFICATION%'
ORDER BY c.table_name, c.column_name;
```

Compliance Architecture Patterns

COMPLIANCE ARCHITECTURE FOR FINANCIAL/HEALTHCARE:



- WAL encryption

AUDIT LOG LAYER

- External, append-only audit storage
- Log shipping (Filebeat/Fluentd → Elasticsearch)
- Long-term retention (1-7 years per requirement)
- SIEM integration for alerting

Common Mistakes

1. **Treating compliance as a one-time project** — it's ongoing
 2. **Blanket encryption of all data** — unnecessary for non-PII, creates overhead
 3. **Not testing erasure procedures** — right-to-erasure fails in production
 4. **Audit logs stored only in PostgreSQL** — DBA can delete them
 5. **No data classification** — can't protect what you haven't identified
 6. **Superuser for application connections** — makes least privilege impossible
 7. **No retention policy enforcement** — data grows indefinitely
 8. **Compliance checkbox approach** — meeting requirements vs. actual security
 9. **No incident response plan** — breach notification requirements not met
 10. **Not documenting controls** — auditors need evidence
-

Best Practices

1. **Separate PII tables** from operational data — enables selective retention
 2. **Implement erasure as pseudonymization** — preserve audit history, remove PII
 3. **Automate retention enforcement** — cron jobs via pg_cron
 4. **Maintain a PHI/PII inventory** — know what personal data you have and where
 5. **Test compliance controls** — quarterly access reviews, annual DR tests
 6. **Document everything** — auditors need written procedures and evidence
 7. **Use defense in depth** — TLS + RLS + column encryption + audit + OS encryption
 8. **Appoint a DPO/Privacy Officer** with technical PostgreSQL knowledge
 9. **Build privacy by design** — default-encrypted, default-minimal access
 10. **Keep PostgreSQL updated** — security patches are compliance-critical
-

Interview Questions

Q1: What is GDPR's right to erasure and how do you implement it in PostgreSQL?

A: Art. 17 requires organizations to erase personal data on request. In PostgreSQL, hard deletion often breaks referential integrity and removes audit history. Best practice: pseudonymization — replace PII with non-identifying values (e.g., email → `erased_USER123@invalid.com`), nullify optional PII fields, delete from PII-only tables, but preserve transactional records (order history, financial records) with anonymized references. Log every erasure event for accountability.

Q2: What PostgreSQL features help meet SOC 2 security criteria?

A: CC6.1 (access control): RBAC with roles, least privilege. CC6.2 (authentication): scram-sha-256, SSL, MFA via LDAP. CC6.3 (network access): `pg_hba.conf`, SSL-only connections, firewall rules. CC6.7 (transmission): SSL/TLS

encryption. CC7.1 (threat detection): pgaudit, log_connections, alerting on anomalies. CC7.2 (vulnerability management): regular PostgreSQL updates, security patches.

Q3: What is PHI under HIPAA and how does minimum necessary apply to PostgreSQL?

A: PHI (Protected Health Information) includes any information relating to health status, treatment, or payment linked to an individual. Minimum necessary means users should access only the PHI required for their role. In PostgreSQL: use RLS to restrict provider access to their patients' records, column-level privileges to hide sensitive fields from support staff, and logging of all PHI access for the audit control.

Q4: What is the difference between anonymization and pseudonymization under GDPR?

A: Anonymization irreversibly removes all personal identifiers — re-identification is impossible. Pseudonymization replaces identifiers with pseudonyms — re-identification is possible with additional information (the mapping). GDPR treats pseudonymized data as still personal data (just lower risk). Anonymized data is outside GDPR scope. In practice, true anonymization of relational data is difficult; pseudonymization is the safer implementation choice.

Q5: How do you handle GDPR data portability (Art. 20) in PostgreSQL?

A: Implement a function that queries all tables containing user data and exports it as a structured format (JSON or CSV). The function should join all relevant tables, decrypt any encrypted fields the user owns, format the data in a human-readable/machine-readable way, and log the export event. The export must be provided within 30 days of request.

Q6: What PCI-DSS requirements relate to PostgreSQL?

A: Req 3: Never store CVV/CVV2; only store masked PAN. Req 6: Protect against vulnerabilities (keep PostgreSQL patched). Req 7: Restrict access to card data (RBAC, least privilege). Req 8: Identify and authenticate users (no shared accounts, scram-sha-256). Req 10: Log and monitor all access to card data (pgaudit on payment tables). Req 12: Maintain security policies (documented access procedures).

Q7: How would you implement automated data retention in PostgreSQL?

A: (1) Define retention rules in a policy table. (2) Create a function that executes retention: DELETE rows older than retention period, anonymize rather than delete where audit history needed. (3) Schedule with pg_cron: `SELECT cron.schedule('...')`. (4) Log each retention run. (5) Alert on failures. Test quarterly by verifying data older than retention period doesn't exist.

Q8: How do you create evidence for a compliance audit?

A: Run queries on: (1) `pg_roles` — list all users and privileges. (2) `pg_hba_file_rules` — authentication configuration. (3) `pg_stat_ssl` — verify SSL usage. (4) `SHOW pgaudit.log` — audit configuration. (5) Query the audit log for the audit period. (6) Run backup verification. (7) Show change management (DDL audit log). Package as a compliance report with timestamps and DBA signature.

Exercises and Solutions

Exercise 1: GDPR Erasure Implementation

```
-- Complete GDPR erasure workflow with verification

BEGIN;
```

```

-- Check user exists and is active
DO $$
DECLARE v_user_id INT := 42;
BEGIN
    PERFORM 1 FROM users WHERE id = v_user_id;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'User % not found', v_user_id;
    END IF;

    -- Execute erasure
    PERFORM gdpr_erase_user(v_user_id);

    -- Verify erasure
    IF EXISTS (
        SELECT 1 FROM users
        WHERE id = v_user_id
        AND email NOT LIKE 'ERASED_%@erased.invalid'
    ) THEN
        RAISE EXCEPTION 'Erasure verification failed for user %', v_user_id;
    END IF;

    RAISE NOTICE 'GDPR erasure for user % completed and verified', v_user_id;
END;
$$;

COMMIT;

```

Exercise 2: SOC 2 Evidence Report

```

-- Generate SOC 2 evidence report
SELECT jsonb_pretty(jsonb_build_object(
    'report_date', now(),
    'generated_by', current_user,

    'user_accounts', (
        SELECT jsonb_agg(jsonb_build_object(
            'username', rolname,
            'is_superuser', rolsuper,
            'has_login', rolcanlogin,
            'password_expires', rolvaliduntil
        ))
        FROM pg_roles
        WHERE rolname NOT LIKE 'pg_%'
        ORDER BY rolname
    ),

    'ssl_configuration', jsonb_build_object(
        'ssl_enabled', (SELECT setting FROM pg_settings WHERE name = 'ssl'),
        'min_protocol', (SELECT setting FROM pg_settings WHERE name =
'ssl_min_protocol_version'),
        'ciphers', (SELECT setting FROM pg_settings WHERE name = 'ssl_ciphers')
    )
);

```

```
    ),  
  
    'audit_configuration', jsonb_build_object(  
        'log_connections', (SELECT setting FROM pg_settings WHERE name =  
'log_connections'),  
        'pgaudit_log', (SELECT setting FROM pg_settings WHERE name = 'pgaudit.log')  
    ),  
  
    'non_ssl_hba_entries', (  
        SELECT jsonb_agg(jsonb_build_object(  
            'type', type,  
            'database', database,  
            'user', user_name,  
            'method', auth_method  
        ))  
        FROM pg_hba_file_rules  
        WHERE type = 'host'  
    )  
);
```

Cross-References

- [01 authentication methods.md](#) — Authentication compliance
- [02 roles and privileges.md](#) — Access control compliance
- [03 row level security.md](#) — Row-level data protection
- [05 encryption.md](#) — Encryption at rest
- [06 auditing.md](#) — Audit logging
- [07 secrets management.md](#) — Credential management compliance